

How Audi improved their chat experience with Generative AI on Amazon SageMaker

by Fabrizio Siciliano, Bruno Pistone, Domenico Capano, Farooq Khan, Francesco Ongaro, Matteo Lanati, Michael Pawelke, Timo Schmidt, and Toaha Umar | on 15 FEB 2024 | in [Automotive](#), [Generative AI](#), [Industries](#) | [Permalink](#) | [Comments](#) | [Share](#)

[Audi AG](#) is a German automotive manufacturer and part of the Volkswagen Group. It has production facilities in several countries including Germany, Hungary, Belgium, Mexico and India. With a strong focus on quality, design and engineering excellence, Audi has established itself as a leading brand in the global luxury car industry that designs, engineers, produces, markets and distributes luxury vehicles.

[Reply](#) specializes in the design and implementation of solutions based on new communication channels and digital media. As a network of highly specialized companies, Reply defines and develops business models enabled by the new models of AI, big data, cloud computing, digital media and the internet of things. Reply delivers consulting, system integration and digital services to organizations across the telecom and media; industry and services; banking and insurance; and public sectors.

Audi, and Reply worked with Amazon Web Services (AWS) on a project to help improve their enterprise search experience through a **Generative AI chatbot**. The solution is based on a technique named **Retrieval Augmented Generation (RAG)**, which uses AWS services such as [Amazon SageMaker](#) and [Amazon OpenSearch Service](#). Ancillary capabilities are offered by other AWS services, such as [Amazon Simple Storage Service](#) (Amazon S3), [AWS Lambda](#), [Amazon CloudFront](#), [Amazon API Gateway](#), and [Amazon Cognito](#).

In this post, we discuss how Audi improved their chat experience, by using a Generative AI solution on Amazon SageMaker, and dive deeper into the background of the essential components of their chatbot, by showcasing how to deploy and consume two state-of-the-art Large Language Models (LLMs), [Falcon 7B-Instruct](#), designed for Natural Language Processing (NLP) tasks in specific domains where the model follows user instructions and produces the desired output, and [Llama-2 13B-Chat](#), designed for conversational contexts where the model responds to user's messages in a natural and engaged way.

How this situation came about with Audi and Reply

For over 3 years, Reply has been helping Audi transition to the cloud. As Audi's internal knowledge base grew rapidly, access to internal documentation became difficult to navigate at times. For example, the difficulties in keeping the pages up to date, the scattering of topics over multiple documents in different locations, the presence of redundant or outdated information. These aspects posed a significant challenge to educating and training activities. Also, the Audi internal ticketing system receives diverse queries from developers, and it often takes them hours to navigate the documentation and fully grasp a topic. This situation has resulted in productivity losses and less-than-optimal service response times.

The following paragraphs provide an overview of the solution, discuss its features, and report the results of the pilot project.

Solution overview

The high-level architecture of the Generative AI chatbot is illustrated in Figure 1 below:

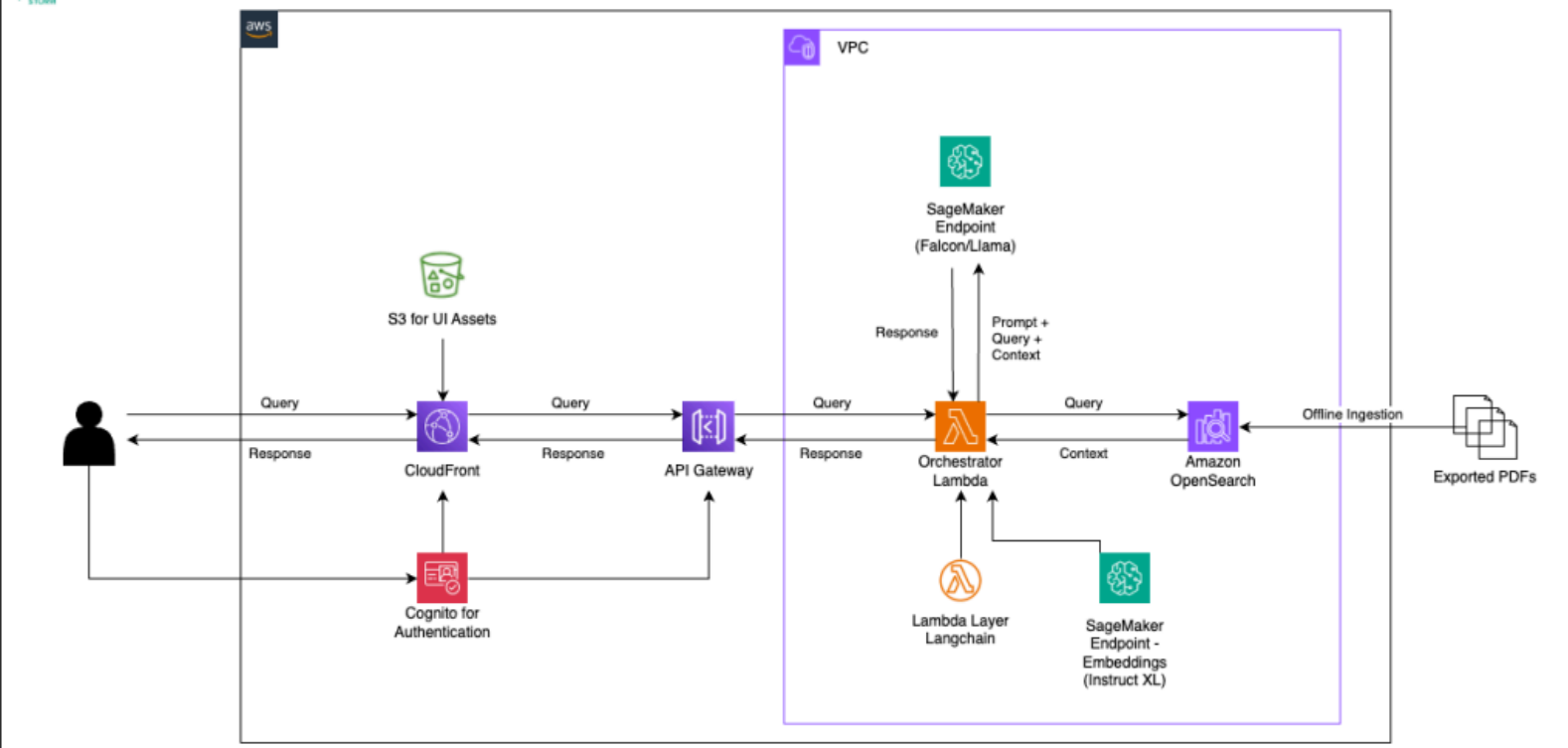


Figure 1: High-Level Architecture of Generative AI Chatbot

The solution workflow can be described in two steps: data ingestion and chatbot inference. These two steps are part of the RAG technique used to power the chatbot solution.

Data Ingestion

In this process, the data to be ingested consists of documents from the Confluence space. An external data ingestion component accesses Confluence using an API key, and converts the documents into a readable text format. The text is then split into smaller chunks of text and tokenized using a [Recursive Character Splitter](#) and tokenizer from the selected LLM.

The dimension of the chunks can vary according to the accepted number of tokens of the LLM, named context window. For the Falcon 7B-Instruct, which has a context window of 2048 tokens, we used a chunk size of 200 with an overlap of 20. In contrast, we used a chunk size of 1000, with an overlap of 200, for the Llama 2 13B-Chat model, as it has a longer context window of 4096 tokens.

These chunks are then fed to the embeddings model and converted into vectors of embeddings, then stored in a vector database where it can be subsequently queried. The querying of the data uses semantic search techniques, described in the next section.

Chatbot Inference

The chatbot, shown in Figure 2 below, consists of the following steps:

- 1. Authentication:** Audi users first log into the User Interface (UI) by authenticating themselves. After successful completion, users are directed to the chatbot UI.
- 2. Querying:** The user can then start querying the chatbot. The queries are transmitted through the API gateway to the Lambda function, where they are then converted into embeddings by the hosted embeddings model.
- 3. LLM Query extraction:** The hosted LLM extracts the most relevant parts of the query.
- 4. Vector Retrieval:** The LLM output is passed to the vector database using the [LangChain](#) Lambda. Based on this query, the database retrieves the most relevant k vectors by using semantic search techniques, to be used as context.

5. **Response Generation:** The context, along with the query, is combined with a prompt that instructs the LLM to only answer if the similarity scores are above a predefined threshold. The LLM then evaluates the generated prompt and generates an output accordingly.

Figure 2: the Audi Chatbot Interface

Semantic Search

Semantic search is a technique that tries to find the most relevant results based on the meaning and context of the query. The main difference between semantic search and traditional keyword search is that the latter tries to match the exact words or phrases in the queries to the retrieved results. In contrast, with the help of a Deep Neural Network (DNN) engine, semantic search features retrievals with the search context and answers the questions in a human-like manner.

To enable semantic search, the documentation to be queried is split into chunks, tokenized, and converted into embeddings. The embeddings are then stored as vectors in a vector database, capable of storing data as high-dimensional vectors. The dimensions of the vectors are dependent on the complexity and granularity of the data. Vector databases enable quick and highly accurate similarity search and retrieval of relevant data based on the input query. Similarity search is enabled by calculating the distance between two vectors, be it cosine search, Euclidean distance, or hamming distance.

Furthermore, the vector database utilizes the approximate K-Nearest Neighbors (KNN) algorithm to cluster the vectors closest in distance. Approximate KNN can be computed using different algorithms, which can be implemented using different engines such as [Non-Metric Space Library](#) (NMSLIB), [Apache Lucene](#), and [Hierarchical Navigable Small Worlds](#) (HNSW).

To summarize how semantic search works, the query is first converted into an embeddings vector using the same function that was used to process the documentation. Then, the approximate KNN algorithm retrieves the k most relevant elements from the vector database, which represents how many “neighbors” we are looking at, by calculating the distance between the query and the elements in the dataset using the embeddings vectors.

Solution details

The key features of the chatbot are shown in Figure 3 and described below:

Figure 3: Key Features of the Chatbot

- **Security:** The chatbot solution is designed to consist of two tiers of security. Isolating the solution with a VPC helps ensure safety and privacy from external actors. The in-house approach is also designed to enable employees and customers to extract insights from their proprietary data, thus helping to reduce the possibility of information leakage to third-party applications.
- **Improved Hallucination Resistance:** Modern Generative AI chatbots are often prone to hallucinations if they are asked a question from outside their training corpus. Our implementation relies on RAG and efficient prompt engineering to help reduce the degree of hallucination as much as possible. Moreover, the chatbot kindly advises the user that no answer can be provided if an adequate confidence level is not met.
- **Low Latency:** Employees often spend a significant amount of time searching for information in their humongous knowledge base, leading to productivity loss. Our chatbot not only reduces the search time from hours to few seconds but is also able to reason and generate unique insights from your data.
- **Multiple Sources Integration:** From Confluence to PDF, from Notion to customized websites, our AI chatbot solution is capable of ingesting data from a variety of sources. The integration flexibility opens up a plethora of opportunities to leverage information from multiple unstructured data sources and generate actionable insights.

SageMaker Endpoints

The LLMs are hosted on Amazon SageMaker endpoints, in particular, Falcon 7B-Instruct runs on a `m1.g5.4xlarge` instance, while Llama2 13B-Chat is powered by a `m1.g5.12xlarge` instance. The deployment is carried out manually by means of an AWS SDK script. This approach assists with flexibility for fine-tuning and a faster development cycle. Comparison details are described in the next paragraphs.

Furthermore, we deployed the HuggingFace [Instructor-XL](#) embeddings model, selected by analyzing the [MTEB Public Leaderboard](#), to convert the documentation as well as the query into embeddings vectors. In this case the runtime is of type `m1.r5.xlarge`, a memory-optimized EC2 instance to ensure that the results are readily available with low latency and in a cost-optimized manner.

Amazon OpenSearch

Amazon OpenSearch is the vector database of choice, a managed service with advanced support for vectors, capable of unburdening the user from the hassles of daily operations.

The choice of Amazon OpenSearch is due to greater flexibility in optimizing chunk size, selection of search algorithm and embeddings model, as well as lower operational costs for the use case.

Lambda Function

The Lambda function acts as an orchestrator, connecting the different components and organizing the flow of information. It uses a [Lambda Layer](#) that implements [LangChain](#), a framework which simplifies and speeds up the creation of Generative AI applications.

CloudFront, API Gateway, and Cognito

The chatbot's frontend is hosted on a serverless architecture using Amazon S3 and CloudFront. User connections are encrypted with TLS and authenticated via Cognito to restrict access.

The API Gateway provides a REST API that connects the frontend to the Lambda function. It facilitates communication between these services while governing usage through throttling and access controls. This infrastructure ensures secure and reliable connectivity that can scale on demand.

Virtual Private Cloud (VPC)

The Lambda function, LLM endpoint, embeddings endpoint, and vector database are all hosted inside an [Amazon Virtual Private Cloud](#) (Amazon VPC). Architecting these inside an Amazon VPC ensures high security of the solution, as each component with network exposure is private with restricted access to the internet. The application's security and privacy aspects are strengthened since the capability of external bad actors to steal data from the vector database or tamper with the endpoints is greatly reduced.

Results

Comparison of the LLMs

A comparison was made between the two LLMs, Falcon 7B-Instruct and Llama-2 13B-Chat with the results documented in the following table. Since Falcon 7B-Instruct has a lower inference time and is cheaper to host than Llama 2 13B-Chat, the Falcon 7B-Instruct model will be used for further experiments, despite having fewer capabilities than the Llama 2 13B-Chat model.

Results of experimentation on Audi Documentation

We used the Falcon 7B-Instruct model for conducting the experiments on the Audi documentation, and the results are detailed in the following table.

While the model performs well on most queries, it sometimes struggles to answer longer queries, for which we can instead use the Llama 2 family of models. Furthermore, the models are unable to answer specific questions involving names, which can be solved using a hybrid search approach, to combine semantic search techniques with keyword search techniques. An average latency of 6 seconds may be considered high for real-time applications, so the solution could be further optimized to get faster responses.

Conclusion

This blog described how the Audi enterprise search experience was improved, thanks to the innovative Generative AI chatbot solution on AWS. The Audi chatbot is designed to help reduce search time from hours to few seconds by providing high fidelity and accuracy in the generated responses.

With the rapid developments in the Generative AI landscape, Reply, Audi, and AWS will continue to empower this solution by adding additional features for security, performance, and cost optimization, and scaling it further by introducing new use cases across multiple verticals within Audi.

To learn more about running your AI/ML and Generative AI experimentation and development workloads on AWS, visit [Amazon SageMaker](#).

TAGS: [Artificial Intelligence](#), [automotive](#), [Customer Solutions](#)

 Like  Share

Comments

[Log in to comment](#)
